DevOps Overview & Git Workflow

Lecture 1

greenwich.edu.vn

UNIVERSITY OF GREENWICH

Alliance with FPT Education

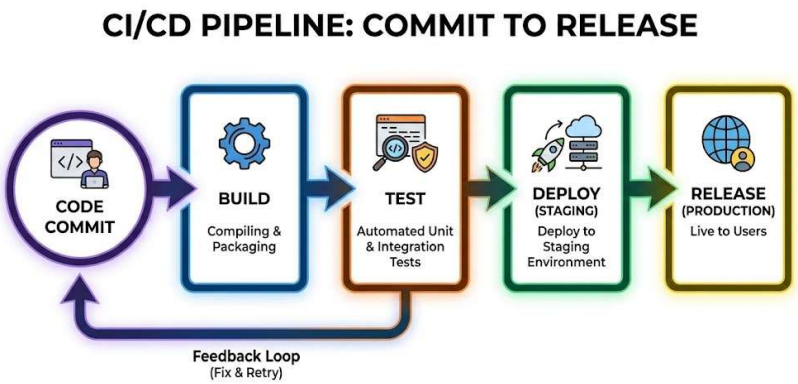
UNIVERSITY OF GREENWICH

Alliance with FPT Education

COURSE INTRODUCTION

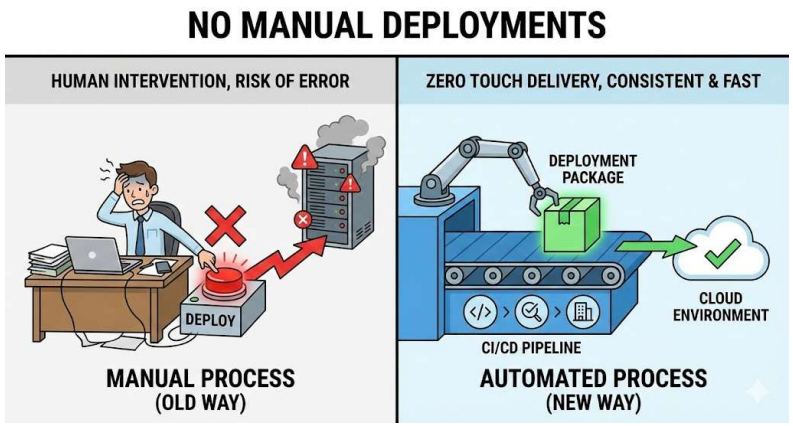
What will we achieve ?

- We will build a fully automated **CI/CD pipeline** where:
  - Developer writes code
  - Code is automatically tested
  - Docker image is built
  - Image is pushed to registry



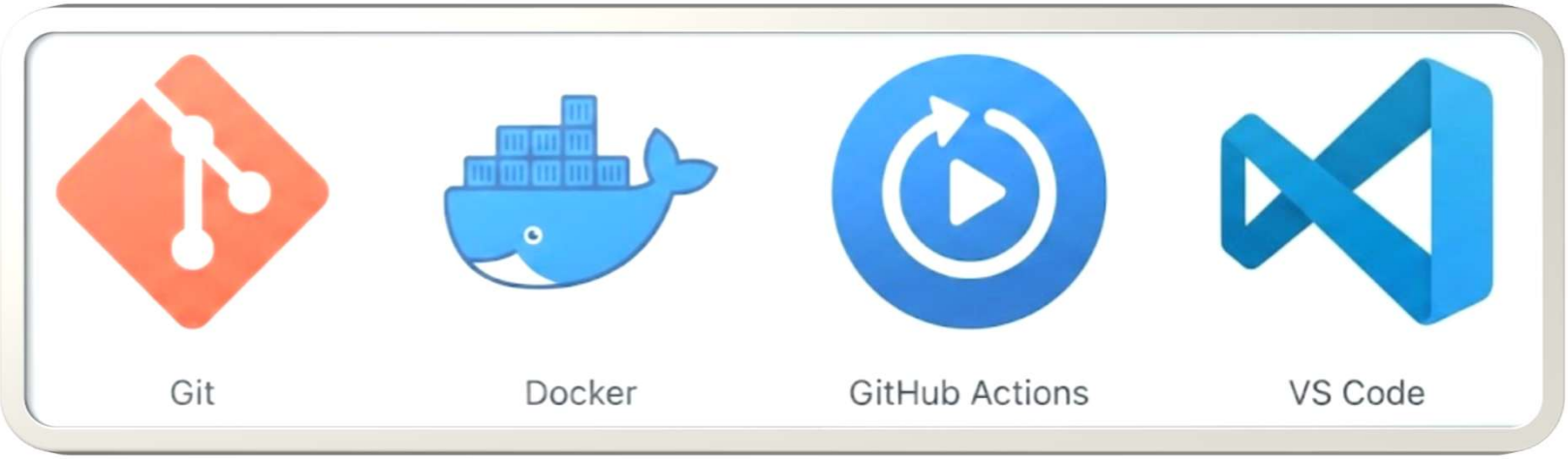
What will we achieve ?

- We will build a fully automated **CI/CD pipeline** where:
  - Deployment happens automatically
  - New version is live in seconds
  - No manual deployments
  - No "**it works on my machine**" problems
  - No human clicking buttons on servers



What will we use ?

- **Tech Stack**



How will you be evaluated ?

Group Project
(*2-3 students/group*)

Create CI/CD Pipeline

Assessment

Slide

Report

Presentation + Q&A

Lesson's Agenda

Section 1: Basic Concepts

Section 2: DevOps Mindset

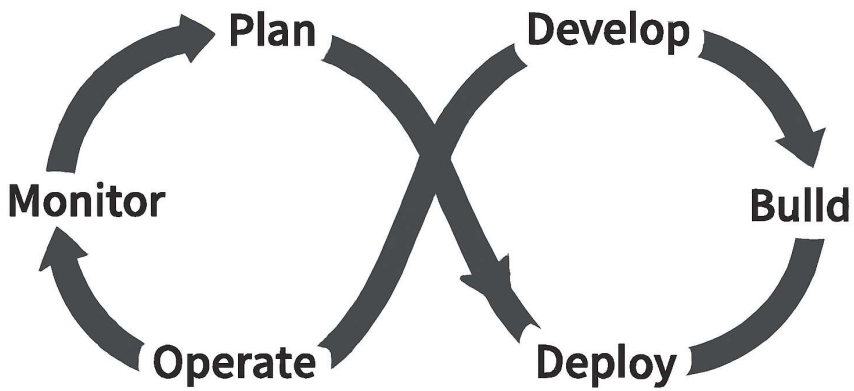
Section 3: Git Refresher

Section 4: Professional Workflow

Section 1: Basic Concepts

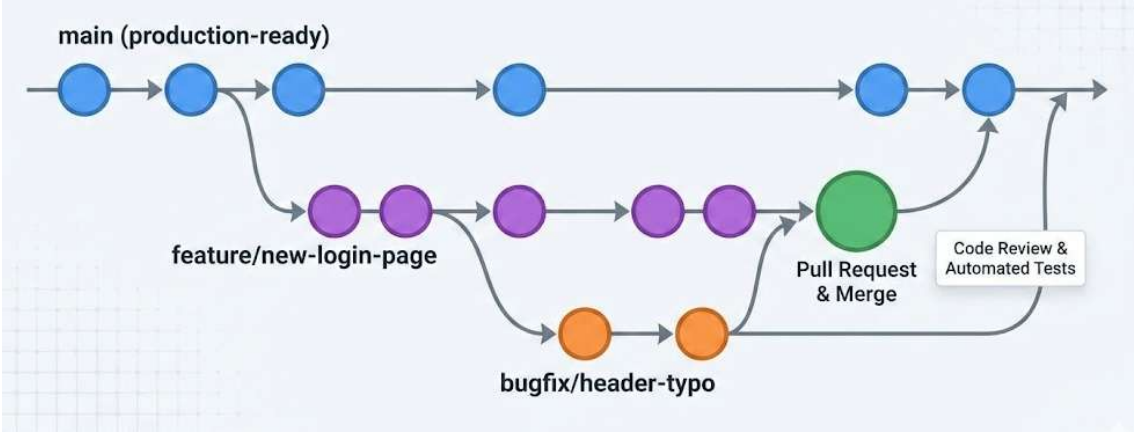
What is DevOps ?

- DevOps is a methodology that bridges the gap between development and operations, enabling faster and more reliable software delivery.
- Focus on automation, collaboration, and continuous improvement.



What is Git Workflow ?

- Git is the version control system that facilitates collaborative development.
- Git workflow helps streamline collaboration among developers by maintaining a structured and manageable code repository.



UNIVERSITY OF GREENWICH

Alliance with FPT Education

Section 2: DevOps Mindset

The Wall of Confusion

- **Development (Dev)** and **Operations (Ops)** were separate silos.
- **Dev Goal:** Change, Speed, New Features.
- **Ops Goal:** Stability, Uptime, Security.
- **Result:** Conflict. Devs toss code over the wall; Ops struggle to deploy it.

The Wall of Confusion

- **Dev**
  - **Goal**: Ship features fast, innovate, change code often
  - **Mentality**: "Let's push it to production ASAP!“
  - **Pain**: "We don't care about infrastructure"
  - **Result**: Write code, throw it over the wall to Ops

- **Ops**
  - **Goal**: Keep systems stable, reliable, and secure
  - **Mentality**: "Do not break production"
  - **Pain**: "Developers keep changing things; risk to availability"
  - **Result**: Focus on uptime, resist frequent changes, push back on fast releases

Defining DevOps

DevOps is **NOT** a tool
(It's not just using Docker or Jenkins)

DevOps is **NOT** a job title
(though companies hire "DevOps Engineers")

Definition: A culture and practice that bridges the gap between Dev, Ops, and QA to deliver software faster and more reliably

The 5 Pillars of DevOps

- **C**ulture
- **A**utomation
- **L**ean
- **M**easurement
- **S**haring

C is for Culture

- No more "*It works on my machine*"
- Cross-functional teams (Dev+Ops work together)
- Developers are responsible for the code even after it is deployed.
- Operations get involved in the design phase.

A is for Automation

Humans make mistakes. Robots do not.

Goal: Automate the "boring" stuff.

Manual Testing ⇨ Automated Unit Tests	Manual Deployment ⇨ Continuous Deployment (CD)	Manual Server Setup ⇨ Infrastructure as Code (IaC)

L is for Lean

Based on Toyota Manufacturing principles.

Small Batch Sizes: Deploy small changes frequently instead of one giant release per year.

Benefit: Easier to debug, lower risk of failure.

M is for Measurement

You cannot improve what you do not measure.

Key Metrics:

MTTR (Mean Time To Recovery): How fast can you fix a crash?	**Lead Time:** How long from "Commit" to "Production"?	**Deployment Frequency:** How often do you ship?

S is for Sharing

Knowledge is power only when shared.

Practice:

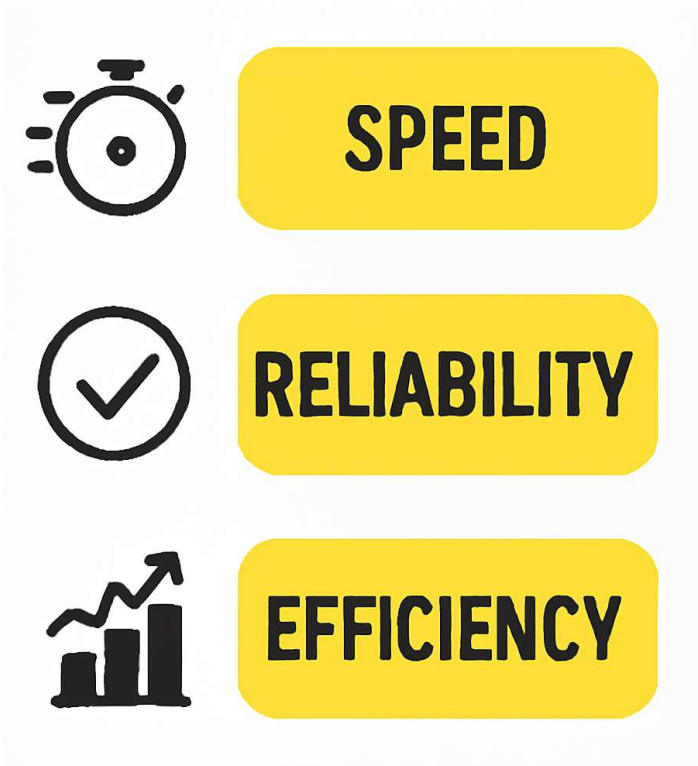
Code Review	**Documentation:** Readme, Wikis	**Blameless Post-Mortems** When things break, focus on the *process*, not the *person*

DevOps Pillars in Real-life

Pillar	Example	Best Practices
Culture	**Microsoft** implemented DevOps practices to foster collaboration and deliver features faster with better integration.	Cross-functional teams, daily standups, and team collaboration tools.
Automation	**Netflix** automates testing, deployment, and continuous integration, enabling multiple daily deployments.	Automate builds, deployments, tests, and infrastructure provisioning.
Lean	**Toyota**'s Lean manufacturing principles were adopted by Target to streamline processes and reduce waste.	Minimize waste, use small batch releases, and continuously improve.
Measurement	**Facebook** uses extensive metrics to monitor performance and ensure optimal service availability.	Use monitoring tools, track key metrics (e.g., lead time, MTTR), and analyze performance.
Sharing	**Amazon** fosters transparency by conducting blameless post-mortems to learn from failures.	Foster transparency, conduct post-mortems, and use documentation tools.

Benefits of DevOps

- Faster delivery cycles and continuous integration.
- Reduced risk of errors in production due to automated testing and deployment.
- Improved collaboration and communication between development and operations teams.



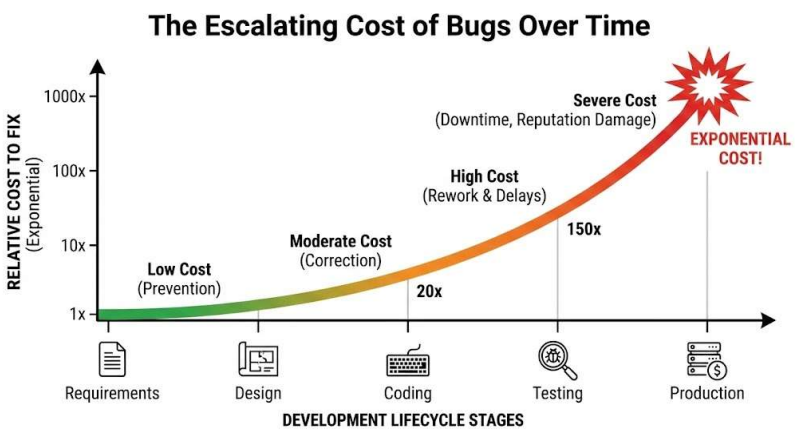
Traditional Development vs DevOps

Aspect	Traditional Development	DevOps
Delivery Speed	Slower, with lengthy development and deployment cycles (Months).	Faster, with **daily** or **weekly** releases (Faster cycle time).
Risk	Higher risk due to lengthy testing and manual deployments.	Lower risk, as **automated tests** and **CI/CD pipelines** ensure code quality.
Collaboration	Silos between Development and Operations teams, leading to friction.	High collaboration between **Dev** and **Ops** with shared responsibility and continuous feedback.
Efficiency	Manual deployment and testing, leading to inefficiency and delays.	**Automation** of build, test, and deployment processes results in higher efficiency.
Incident Response	Slow incident response, often requiring manual fixes.	**Fast recovery**, using metrics and monitoring tools to reduce **MTTR** (Mean Time to Recovery).
Feedback Loops	Long feedback loops; often, feedback occurs only after deployment.	**Short feedback loops**, allowing teams to iterate and deploy quickly.

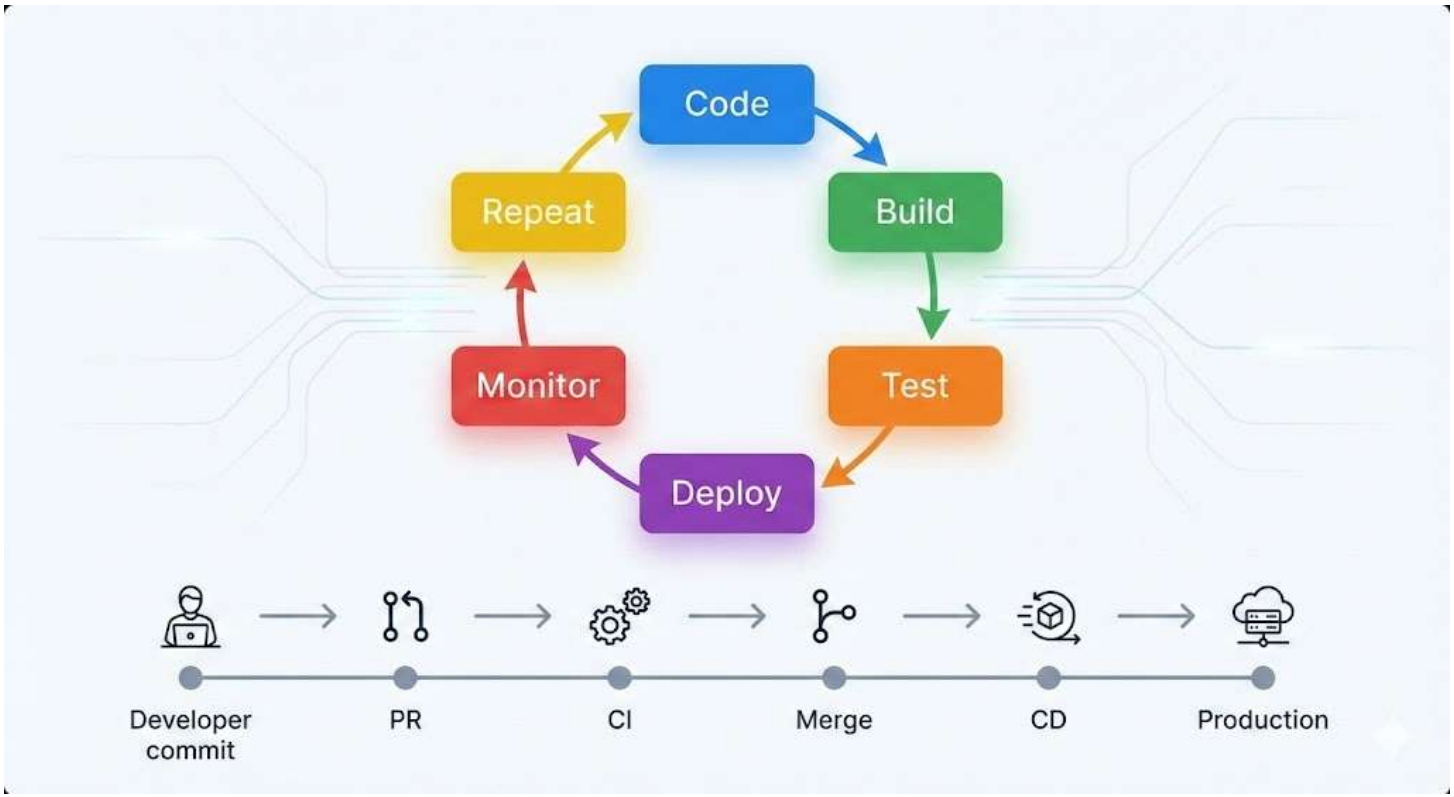
"Shift Left" Testing

- **Traditional testing:** Testing occurs at the end of the development cycle.

- **DevOps:** Testing happens early in the process (Shift Left), which ensures that issues are detected and fixed earlier, reducing costs and improving quality.



CI/CD Lifecycle



UNIVERSITY OF GREENWICH

Alliance with FPT Education

Section 3: Git Refresher

Why Git ?

Version Control: History of all changes. "**Undo**" button.

Distributed: Every developer has a full copy of the code.

Collaboration: Merging work from multiple people.

How Git Works Locally ?

Working Directory: Where you edit files.

Staging Area: Where you prepare files for a commit (git add).

Repository: The permanent history (git commit).

The Survival Kit

`git status` : Where am I?

`git add .` : Stage changes.

`git commit -m "message"` : Save snapshot.

`git push` : Upload to GitHub.

`git pull` : Download updates.

The importance of .gitignore

Tells Git which files to **ignore**.

What to ignore?

node_modules/ (Dependencies - huge!).
.env (Secrets/Passwords - dangerous!).
dist/ or **build/** (Generated binaries).

Tip: Always create .gitignore before your first commit.

Commit Messages Matter

Bad: "*fix*", "*update*", "*wip*", "*bug*".

Good: "*fix: resolve login timeout issue*", "*feat: add payment gateway*".

Why? Your teammates need to understand history without reading code line-by-line.

Working Alone vs Working Together

Solo: You push directly to main. Fast, but risky.

Team: If everyone pushes to main, conflicts happen instantly. The code breaks.

Solution: Branching Strategies.

UNIVERSITY OF GREENWICH

Alliance with FPT Education

Section 4: Professional Workflow

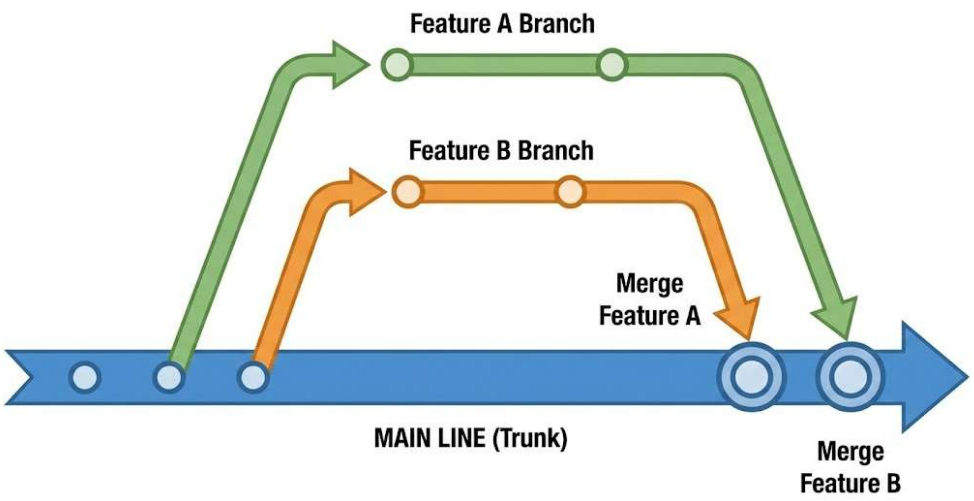
The #1 Rule of Teamwork

- **Never commit directly to Main**
- The **main** (or **master**) branch is for production-ready code only.
- If main is broken, the team cannot deploy.

The Feature Branch Strategy

- For every new task (feature, bugfix), create a new branch.

- **Naming Convention:**
  - feature/login-page
  - fix/header-alignment
  - docs/update-readme



Step 1 - Branching

```
# Update local main
git checkout main
git pull origin main

# Create new branch
git checkout -b feature/new-feature
```

- **Note:** Always pull the latest code before creating a branch

Step 2: Open a Pull Request

You pushed your branch to GitHub. Now what?

Definition: A Pull Request is a request to merge your branch into main.

It is a **Quality Gate**. It forces a pause for review.

Step 3: Code Review

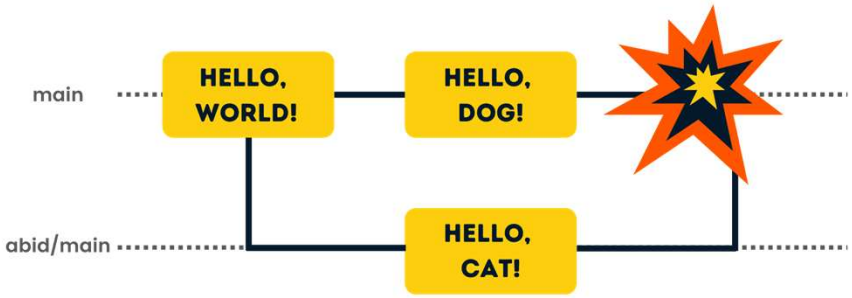
- Teammates read your code diff.
- They look for: Logic errors, messy code, missing tests.
- **Mindset:** Code Review is not criticism; it is mentorship and quality control.
- You must fix the issues raised before merging.

Step 4: Merge

- Once the PR is Approved (**Green Tick**).
- Click "**Squash and Merge**" (keeps history clean).
- Delete the feature branch.
- The code is now in **main.**

Understanding Merge Conflicts

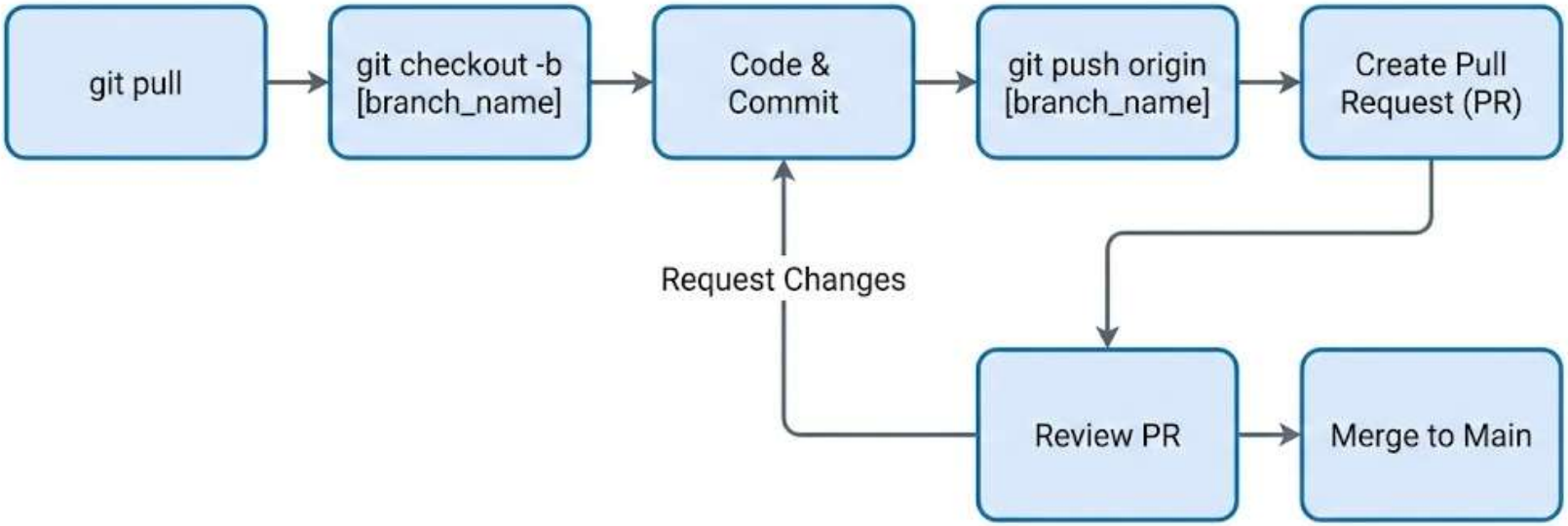
- **Scenario:**
  - *Alice* changes Line 10 of *index.js*
  - *Bob* also changes Line 10 of *index.js*
- Git does not know which line is correct.
- Git pauses and asks humans to decide.



How to fix Conflicts ?

- Open the file (VS Code highlights conflicts clearly).
- Look for **<<<<<<< HEAD** and **>>>>>>> feature**.
- Choose "**Accept Current**" or "**Accept Incoming**".
- Save file.
- **git add .** and **git commit.**

Recap: The Lifecycle



UNIVERSITY OF GREENWICH
Alliance with FPT Education

Final Tips

1

Commit often (small saves)

2

Write clear commit messages

3

Don't work on one branch for 2 weeks

Merge hell!

4

Review your own PR before asking others

UNIVERSITY OF GREENWICH

Alliance with FPT Education